

FPGA Tick-to-Trade — Implementation Results

June 2026

Synthesis & place-and-route results, with a baseline for tracking optimisation.

This document records real Vivado synthesis + implementation (placed & routed) results for `tick_to_trade_top`, so successive optimisations can be compared. Reports live in `results/synthesis/` and `results/implementation/`.

Run B0 — Baseline (registered order-book array)

Item	Value
Target device	xcu200-fsgd2104-2-e (Alveo U200 / UltraScale+ VU9P — AWS F1 class)
Tool	Vivado 2025.x, out-of-context (no physical pin constraints)
Target clock	200 MHz (5.000 ns period)
Flow	synth_design → opt → place → route (fully routed)

Timing — **FAIL** at 200 MHz

Metric	Value
Worst Negative Slack (WNS)	−2.373 ns
Failing endpoints	41,206
Total Negative Slack (TNS)	−51,387 ns
Max achievable clock	≈ 135 MHz (1/(5.0+2.373) ns)
Implied tick-to-trade	39 cycles × 7.37 ns ≈ 290 ns (not 195 ns)

Utilisation (placed) — tiny footprint

Resource	Used	Available	%
CLB LUTs	36,057	1,182,240	3.05
CLB Registers	37,672	2,364,480	1.59
CARRY8	265	147,780	0.18
Block RAM Tile	0	2,160	0.00
DSPs	0	6,840	0.00

Power & DRC

Total on-chip power	2.844 W (dynamic 0.371 W, static 2.473 W; low confidence)
Routing	fully routed (52,563 / 52,563 routable nets)
DRC	2 critical warnings: NSTD-1, UCIO-1 (unconstrained I/O — expected in OOC flow with no pin constraints; benign for F1-shell integration)

Critical path & root cause

Source	u_book/entries_reg[7] [shares] [8]
Destination	u_book/entries_reg[219] [shares] [18]
Data delay	7.036 ns — route 5.969 ns (85%) , logic 1.067 ns, 11 logic levels

Root cause: the order-book `entries[]` array (256×130 bits) synthesised to **37 K flip-flops (BRAM = 0)** instead of memory, and the RESCAN FSM reads *all* entries combinatorially. The huge fan-out of that register file makes the path **route-dominated (85%)**, so it misses 200 MHz badly. Logic is fine (<1.1 ns); the problem is physical (FF array + combinational scan).

Planned fix — Run B1 (BRAM-based order book)

Convert `entries[]` to a true **BRAM with synchronous (registered) reads**. The RESCAN FSM already walks one entry per cycle, so a 1-cycle BRAM read drops in naturally; the Add fast-path stays $O(1)$ (writes + level-insert from the message, no read). This removes both the 37 K-FF array and the route blow-up and should close 200 MHz.

Correction 1 — Run B1 (BRAM order book, measured)

The BRAM order book was implemented and re-run through synth + implementation on the VU9P. **Outcome: the resource/route problem is fixed, but timing still fails — for a new reason.**

Metric	B0 (FF array)	B1 (BRAM)	Change
CLB Registers (FF)	37,672	4,308	−89% ✓
CLB LUTs	36,057	6,790	−81% ✓
Block RAM Tile	0	2	inferred ✓
DSPs	0	0	—
Failing endpoints	41,206	3,612	−91% ✓
Total Negative Slack	−51,387 ns	−2,522 ns	−95% ✓
Worst Negative Slack	−2.373 ns	−4.563 ns	worse
Total on-chip power	2.844 W	2.607 W	−8%
Timing @ 200 MHz	FAIL	FAIL	not yet closed

What worked: `entries[]` now infers block RAM (2 tiles) instead of 37 K flip-flops; LUTs/FFs collapsed ~85%; the route-dominated FF-array path is gone — failing endpoints and total violation both dropped ~90–95%. The original root cause is resolved.

New critical path (B1):

Source	<code>u_book/bid_lv_reg[cnt][0]</code> (level cache)
Destination	<code>risk_reason[1]</code> (output pin / OBUF)
Data delay	5.168 ns — logic 2.494 ns (48%), route 2.673 ns (52%), 14 levels, 5×CARRY8

Why WNS got worse: this is no longer a routing problem — it is a long *combinational* chain: book level cache → strategy (depth-weighted cross-multiply) → `risk_check` (price-collar subtract, position add) → straight to the decision/`risk_reason` output pins, with no register in between. `risk_check` was added as a *zero-latency combinational* gate (to preserve 195 ns); once the FF-array path was removed, this chain became the worst path.

Planned fix — Run B2 (register the risk/decision stage)

Make `risk_check` a **1-cycle pipeline** (register its outputs) and register the decision output ports. This splits book → strategy → risk → pin into short segments. Cost: **+1 cycle** (tick-to-trade 39 → 40 cycles, 195 → **200 ns**); expected to finally close 200 MHz (the worst path is ~9.6 ns of logic+route, splittable by one register).

Status: the B2 register stage is now *implemented* in `tick_to_trade_top` (and `multi_symbol_top`) and verified in simulation — tick-to-trade is now 40 cycles = **200 ns**. It has *not yet been re-synthesised*, so the `results/implementation/` reports still show the B1 numbers (−4.563 ns, dest `risk_reason`). Re-run synth+impl on `tick_to_trade_top` to populate the real B2 timing.

Correction 2 — Multi-symbol engine ($\times 4$), measured

`multi_symbol_top` (NSYMBOLS=4: four per-symbol books routed by `stock_locate`) was synthesised + implemented on the VU9P.

Metric	single-sym B1	multi-sym ($\times 4$)	Note
Block RAM Tile	2	8	exactly $4\times$ — one BRAM book/symbol ✓
CLB LUTs	6,790	19,942	$\sim 3\times$ (shared strategy/risk)
CLB Registers (FF)	4,308	8,984	$\sim 2\times$
WNS	-4.563 ns	-5.362 ns	worse
Failing endpoints	3,612	10,899	—
Power	2.607 W	2.869 W	—
Routing	clean	fully routed, 0 errors	✓

Confirms the BRAM book scales: 4 symbols $\rightarrow$ 8 BRAM tiles (4×2), no FF explosion. **Worst path** (`gbook[3] bid_lv  $\rightarrow$  m_axis_tvalid`, 18 levels, $8\times$ CARRY8) is the same combinational book $\rightarrow$ strategy $\rightarrow$ risk $\rightarrow$ pin chain, *lengthened* by the 4-way active-symbol output mux. This run pre-dates the register fix; the fix has since been applied to `multi_symbol_top` too (re-run pending).

Run history

Run	Change	WNS	Status
B0	registered FF order-book array	-2.373 ns	FAIL (route-bound)
B1	<code>entries[]</code> $\rightarrow$ BRAM	-4.563 ns	FAIL (combinational chain)
M1	multi-symbol $\times 4$ (BRAM, comb. risk)	-5.362 ns	FAIL (chain + sym mux)
B2	register risk/decision stage	TBD	RTL done (200 ns sim); <i>re-run synth</i>